

KMP

Knuth–Morris–Pratt

Análise de Complexidade de Tempo

Pedro Garcez · Rafael Ferraz · Denys Leonardo · Felipe Cavalcanti

Teoria da Computação · Prof. Daniel Bezerra · CESAR School

Python + Java

Agenda

1

O Problema: busca de padrão em string

2

Como funciona o KMP — tabela LPS e busca

3

Classificação assintótica: O , Ω , Θ

4

Análise de casos: melhor, médio e pior

5

Metodologia experimental

6

Resultados: Python e Java

7

Aplicabilidade, classe P e reflexão final

O Problema: Busca de Padrão

Dado um texto **T** (tamanho n) e um padrão **P** (tamanho m), encontrar todas as ocorrências de **P** em **T**.

T = ABABDABAC**ABABCABAB**

P = ABABCABAB

✓ Ocorrência encontrada na posição 10

Busca Ingênuo

$O(n \cdot m)$

pior caso

KMP

$O(n + m)$

qualquer entrada

Rabin-Karp

$O(n + m)^*$

**médio; pior $O(n \cdot m)$*

Como Funciona: a Tabela LPS

LPS = Longest Proper Prefix which is also Suffix

Pré-processa o padrão e indica quanto avançar quando há uma falha, sem retroceder o ponteiro do texto.

i	0	1	2	3	4	5	6	7	8
P[i]	A	B	A	B	C	A	B	A	B
LPS	0	0	1	2	0	1	2	3	4

Complexidade: $O(m)$ — cada caractere do padrão é processado no máximo duas vezes. A LPS permite deslocar o padrão sem retroceder no texto.

Classificação Assintótica

Big-O

$$O(n + m)$$

Limite superior. O KMP nunca realiza mais que $n + m$ comparações — independente da entrada.

Big- Ω

$$\Omega(n + m)$$

Limite inferior. É preciso ler ao menos o texto inteiro (n) e construir a LPS completa (m).

Big- Θ

$$\Theta(n + m)$$

Como $O = \Omega$, então Θ existe. KMP é assintoticamente ótimo: melhor, médio e pior coincidem.

O KMP é um dos poucos algoritmos onde $O = \Omega = \Theta$ — robusto contra qualquer tipo de entrada.

Análise de Casos

Melhor Caso

$\approx n$ comparações

$T = \text{"aaa... a"} \quad P = \text{"baaa... a"}$

O 1º caractere do padrão nunca aparece no texto. A LPS nunca é consultada — 1 comparação por posição.

Caso Médio

$O(n + m)$

Texto e padrão aleatórios

Distribuição uniforme sobre alfabeto de 10 letras. Falhas parciais são raras; retrocesso esporádico.

Pior Caso

$\approx 2n$ comparações

$T = \text{"aaa... a"} \quad P = \text{"aa... ab"}$

$LPS = [0, 1, \dots, m-2, 0]$. Combina $m-1$ chars, falha no último e retrocede — repetidamente.

Metodologia Experimental

Protocolo de Coleta

Rodadas cronometradas	30 / cenário
Aquecimento (warmup)	5 (descartadas)
Tamanhos (n)	1k / 10k / 100k
Tamanho do padrão (m)	1% de n
Estatística	média + DP amostral

Ambiente de Execução

Processador	i7-13650HX
Memória RAM	16 GB DDR5
Sistema Op.	Windows 11
Python / Java	3.13 / JDK 21
Medição	perf_counter / nanoTime

Processos em segundo plano encerrados durante as medições para minimizar ruído.

Resultados — Python

Caso	n = 1.000	n = 10.000	n = 100.000
Melhor	74,63 ± 10,19	799,20 ± 108,18	8.003,94 ± 1.091,06
Médio	101,30 ± 86,30	806,87 ± 123,45	9.515,19 ± 3.397,53
Pior	101,58 ± 12,82	1.024,84 ± 159,15	11.194,91 ± 603,50

μs (microssegundos). Crescimento linear confirmado: **×10 em n → ×10 no tempo**. Ordem esperada: melhor < médio < pior; o pior é $\approx 1,40\times$ mais lento que o melhor (retrocesso via LPS).

Resultados — Java

Caso	n = 1.000	n = 10.000	n = 100.000
Melhor	21,72 ± 1,23	51,12 ± 11,93	114,73 ± 28,69
Médio	8,67 ± 7,91	69,12 ± 44,58	250,86 ± 36,36
Pior	10,40 ± 8,09	84,45 ± 43,69	222,30 ± 37,44

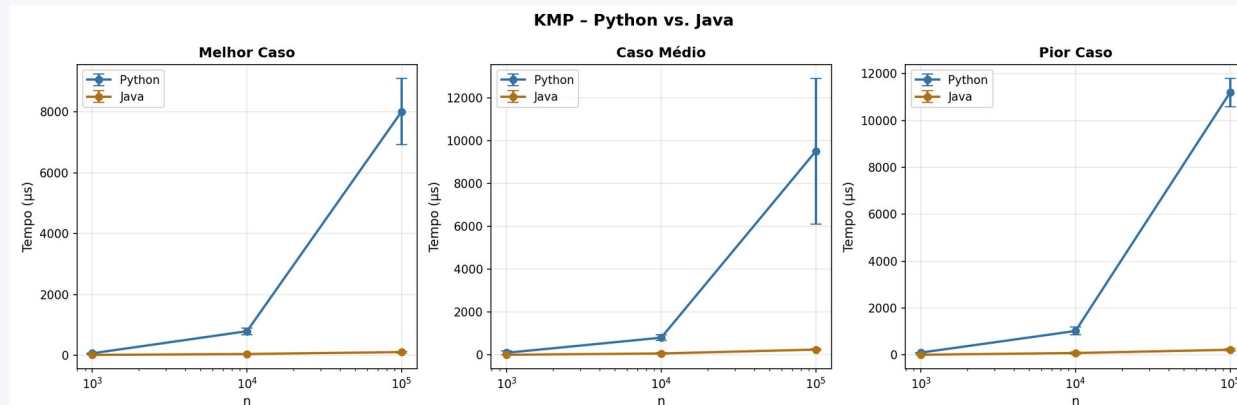
Com entradas idênticas às do Python, o pior caso agora supera corretamente o melhor — a anomalia anterior sumiu. O caso médio virou o mais lento: seu alfabeto variado gera comparações reais e desvios pouco previsíveis (branch prediction e cache piores), enquanto melhor e pior usam texto de um único caractere. Tempos em μ s são sensíveis a ruído — daí os desvios altos em n pequeno. Crescimento $O(n+m)$.

Python vs. Java (n = 100.000)

≈ 70× Melhor
Java mais rápido

≈ 38× Médio
Java mais rápido

≈ 50× Pior
Java mais rápido



Java usa código nativo (JIT) e arrays de char primitivos; Python paga overhead de interpretação e objetos Unicode.

Aplicabilidade



Editores de Texto

Busca e substituição — base do grep e do Ctrl+F.



Bioinformática

Busca de sequências em genomas de bilhões de pares de base.



Segurança

Detecção de padrões em IDS, antivírus e firewalls.



Redes

Inspeção profunda de pacotes (DPI) para filtros de conteúdo.



Compressão

Relacionado ao LZ77 — base de ZIP, GZIP e DEFLATE.



Limitação

Apenas busca exata. Aproximada → Levenshtein ou BK-Tree.

Reflexão Final: Classe P e NP

? KMP pertence à classe P

Resolve a busca exata em tempo linear $O(n + m)$ — polinomial no tamanho da entrada.

Existe algoritmo determinístico eficiente para qualquer entrada.

$$\Theta(n + m) \subset DTIME(n) \subset P$$

? Problemas relacionados

Substring Comum Mais Longa

P — DP $O(n \cdot m)$

Busca Aproximada (Edit Dist.)

P — DP $O(n \cdot m)$

Expressões Regulares

P — Autômatos

Menor Superstring

NP-completo

A intratabilidade surge só em variantes de otimização sobre múltiplos padrões/strings, como o Menor Superstring.

Conclusões

- KMP resolve busca de padrão em $\Theta(n + m)$ — ótimo em todos os casos.
- A tabela LPS é a chave: desloca o padrão sem retroceder no texto.
- Java foi 38×–70× mais rápido que Python para $n = 100.000$.
- Resultados confirmaram crescimento linear nas duas linguagens.
- Pertence à classe P, com ampla aplicabilidade prática.